Day 157: Hybrid Recommenders: Combining Neighbours, Factors and Content

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 4 | Complete study guide

Watch: Maciej Kula - Hybrid Recommender Systems in Python (PyData) | Hybrid Recommender Systems (eggie545)

Script: `day157_hybrid_recommenders.py` · every number in this guide is copied by program from `day157_run_output.txt`, the real output of that script.

Each model of the week has a blind spot: neighbours and factors know nothing about a new film, and content profiles are weak once ratings exist. Today they are combined in three ways, all on Day 154's split. A linear blend fitted on validation brought test RMSE from 0.9115 (ALS alone) to 0.9039 with three models and 0.8973 with five. Letting the blend weights depend on how many ratings a film has improved films with 0–5 ratings from 1.0487 to 0.9930. Putting the content inside the factor model gave one model that matched ALS on familiar films (test 0.9104) and could still score new ones (1.0314 against the 1.0567 fallback), though Day 156's content profiles were better there (1.0222).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

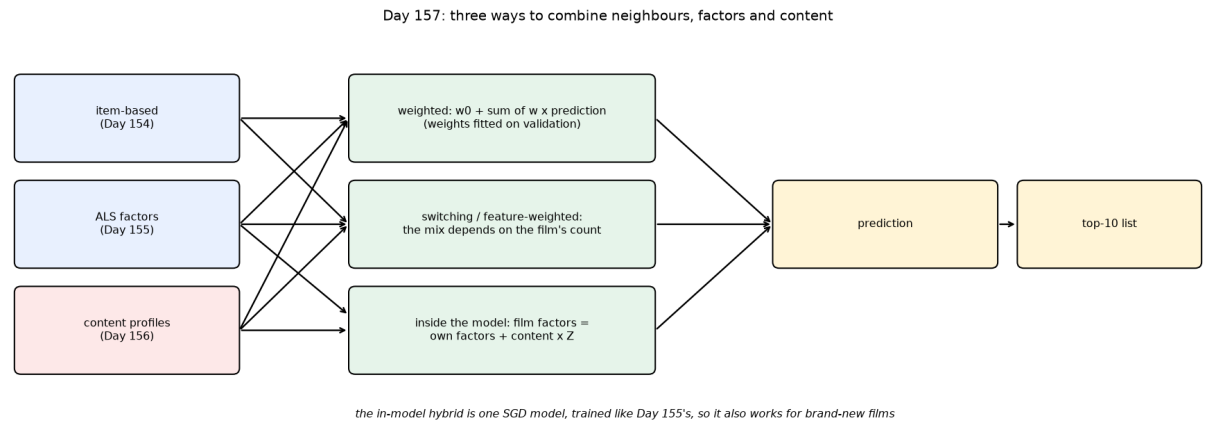
A **hybrid recommender** combines two or more recommenders. The three classic designs are: **weighted** (a weighted sum of their predictions), **switching** (use one or the other depending on the situation) and **feature combination** (one model that takes both kinds of information as input).

Stacking (called blending in the Netflix Prize) learns the weights of a weighted hybrid from data the models were not trained on: here, the validation ratings.

REAL-WORLD SCENARIO

A hospital combines a lab test, a scan and a doctor's examination. Each is wrong in different cases, so the combined judgement is better than any one, and for a brand-new kind of patient the doctor's examination may be all there is.

SYSTEM DIAGRAM



Three ways to combine Days 154, 155 and 156: weighted, switching or feature-weighted, and inside the model.

Today's part for the week's build

`day157_exercise.py` is today's parts (the blend, the feature-weighted design, the switch, and the factor model with content inside), checked by hand.

ACTUAL OUTPUT

```
y = 1 + 2 a + 3 b exactly: stack_fit gives [1.0, 2.0, 3.0]
blend of predictions 4 and 3 with weights (0.5; 0.6, 0.3): 0.5 + 2.4 + 0.9 = 3.8; stack_predict
    gives 3.8000
fw_design for a film with 0 ratings (log 1 = 0) and predictions 4, 3: [0.0, 4.0, 3.0, 0.0, 0.0]
    (0, the two predictions, then 0, 0)
switch at 10 ratings, films with 0, 5 and 20: rich 4, poor 3 -> 3, 3, 4; switch_predict gives
    [3.0, 3.0, 4.0]
in-model hybrid, 8 users x 6 films, films 4-5 never seen, penalty 0.0: their own factors stay 0.0;
    largest error on seen films 0.001, on unseen films, from genre alone 0.474
in-model hybrid, 8 users x 6 films, films 4-5 never seen, penalty 0.1: their own factors stay 0.0;
    largest error on seen films 0.010, on unseen films, from genre alone 0.019
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The ingredients: five models, and whether their mistakes differ

DEFINITION — start here, no prior knowledge assumed

A blend can only help if the models are wrong in *different* places. The **correlation of errors** measures this: 1 means two models make exactly the same mistakes, lower means there is something to gain by combining them.

Each model is trained twice: once without the validation ratings (its validation predictions are used to fit the blend), and once on all 80,000 training ratings (its test predictions are blended with those weights).

REAL-WORLD SCENARIO

Two weather forecasters who always agree are no better together than apart; two who are wrong on different days are.

IMPLEMENTATION

```
def ingredients(u, i, r, X, nu, ni, fit, ev):
    R, M = d154.to_matrix(u[fit], i[fit], r[fit], nu, ni)
    mb, bu, bi = d154.fit_biases(R, M, LU, LI); B = mb + bu[:, None] + bi[None, :]
    Xr = (R - (R.sum(1) / np.maximum(M.sum(1), 1))[:, None]) * M
    S, _ = d154.similarity(Xr, M, shrink=ITEM["shrink"])
    c, _ = d154.predict_item_knn((R - B) * M, M, S, u[ev], i[ev], ITEM["k"])
    F, _ = d155.als_mf(R, M, ALS["k"], ALS["lam"])
    mu0, bu0, _ = d154.fit_biases(R, M, LU, 1e12); T = (R - mu0 - bu0[:, None]) * M
    W = d156.fit_profiles(X, T, M, PROF["lam"], prior=d156.fit_shared_profile(X, T, M,
        PROF["shared"]))
    Tb = (R - B) * M
    Wb = d156.fit_profiles(X, Tb, M, PROF["lam"], prior=d156.fit_shared_profile(X, Tb, M,
        PROF["shared"]))
    b = B[u[ev], i[ev]]
    P = np.c_[b, b + c, d155.mf_predict(F, u[ev], i[ev]),
              mu0 + bu0[u[ev]] + d156.content_predict(W, X, u[ev], i[ev]), b +
                  d156.content_predict(Wb, X, u[ev], i[ev])]
    return P, M.sum(0)[i[ev]], dict(B=B, F=F, W=W, Wb=Wb, M=M, mu0=mu0, bu0=bu0)
```

SPECIFICATION

Models	Day 154's bias model and item-based neighbours (k 40, shrink 800); Day 155's ALS (100 factors, λ 0.125); Day 156's content profiles alone and on top of the bias model; every setting as chosen on those days
Data	Day 154's split: 72,000 + 8,000 validation ratings for blending, 20,000 test ratings

ACTUAL OUTPUT

```
each model trained twice: without the validation ratings (to fit blends) and on all 80,000 (60 s)
                   validation     test
        bias model      0.9511   0.9401
       item-based       0.9242   0.9153
       ALS factors      0.9238   0.9115
  content profiles      1.0073   0.9901
    bias + content      0.9349   0.9222
correlation of test errors:
                   bias model item-based ALS factor content pr bias + con
        bias model      1.000      0.964      0.975      0.907      0.986
       item-based       0.964      1.000      0.973      0.878      0.961
       ALS factors      0.975      0.973      1.000      0.906      0.967
  content profiles      0.907      0.878      0.906      1.000      0.920
    bias + content      0.986      0.961      0.967      0.920      1.000
Actual output from this lesson's run.
```

What it means

The ratings-based models make very similar mistakes: item-based and ALS errors correlate 0.973. Content profiles are the most different (correlation 0.878 with item-based), which is why they can add something to a blend despite being the weakest model on their own.

WHY CHOOSE THIS?

Reusing models already built and tuned costs nothing extra but the blend.

WHY NOT THIS? (TRADEOFFS)

Every model must be trained twice, and all of them must be kept and run in production.

2. Weighted hybrid: a linear blend fitted on validation

DEFINITION — start here, no prior knowledge assumed

The blend predicts $w_0 + w_1 \times$ (model 1's prediction) $+ w_2 \times$ (model 2's) + ...; the weights come from least squares on the validation ratings.

Three checks keep it honest: a column of **pure noise** added to the blend (it should get almost no weight); the same blend with weights fitted on the test set itself (a **leak**, shown only to measure how optimistic that would be); and weights fitted on one half of validation, scored on the other half.

REAL-WORLD SCENARIO

A committee vote where members who have been right more often in past meetings get more votes.

MATHEMATICS

$$blend = w_0 + \Sigma_m \, w_m \, p_m; \quad w = argmin \, \Sigma_{validation} \, (r - blend)^2$$

IMPLEMENTATION

```
def stack_fit(P, y):
    A = np.c_[np.ones(len(y)), P]
    return np.linalg.lstsq(A, y, rcond=None)[0]
def stack_predict(P, w):
    return w[0] + P @ w[1:]
```

ACTUAL OUTPUT

```
                                          blend of    test   weights (intercept first)
             item-based + ALS, plain average (no fitting)  0.9073
                               item-based + ALS factors  0.9061   -0.336 +0.467 +0.615
                item-based + ALS factors + content profiles  0.9039   -0.488 +0.445 +0.555 +0.125
item-based + ALS factors + content profiles + bias + content  0.9039   -0.456 +0.415 +0.519 +0.097 +0.086
bias model + item-based + ALS factors + content profiles + bias + content  0.8973   -0.250 -0.924 +0.471 +0.720 +0.017
    +0.772
three-model blend: model weights sum to 1.126; plus a noise column: noise weight -0.0148, test 0.9044; weights fitted on
    test (a leak): 0.9031; fitted on half of validation, scored on the other half: 0.9108
five-model blend: model weights sum to 1.057; plus a noise column: noise weight -0.0124, test 0.8977; weights fitted on test
    (a leak): 0.8963; fitted on half of validation, scored on the other half: 0.9012
Actual output from this lesson's run.
```

What it means

Averaging item-based and ALS already beat both (0.9073); fitted weights did slightly better (0.9061), and adding content profiles helped more (0.9039) than its weak solo score would suggest, because its errors differ. The five-model blend reached 0.8973 with odd-looking weights: -0.924 on the bias model and +0.772 on bias + content, two models whose errors correlate 0.986. A likely reading, not tested here: the blend uses their difference, the content correction, and amplifies it.

The checks: noise got weight -0.0124 in the five-model blend and test barely moved (test 0.8977); fitting on the test set would have reported 0.8963, only slightly optimistic; and fitted on half of validation, the five-model blend still beat the three-model one on the other half (0.9012 against 0.9108). The five-model gain is not a fluke of one fit.

WHY CHOOSE THIS?

Simple, fast, and almost always better than the best single model; the weights show what each model contributes.

WHY NOT THIS? (TRADEOFFS)

With nearly identical models the weights become large and opposite; a blend needs its own held-out data.

3. Switching and feature-weighted hybrids: how much is known about the film

DEFINITION — start here, no prior knowledge assumed

A **switching hybrid** uses the content model for films with fewer than a threshold number of ratings and the ratings-based blend otherwise; the threshold is chosen on validation.

Feature-weighted linear stacking (Sill and others, 2009, from the Netflix Prize) makes every blend weight a straight line in a feature of the film, here log(1 + its number of training ratings), so the mix can slide smoothly from content to ratings as a film gathers ratings.

REAL-WORLD SCENARIO

A new restaurant is judged by its menu and chef; an established one by its reviews. In between, both count.

MATHEMATICS

$$\textit{switch: pred} = \textit{content if } n_i < t, \textit{ else blend;} \quad \textit{feature-weighted: } w_m(i) = a_m + c_m \log(1 + n_i)$$

IMPLEMENTATION

```
def fw_design(P, n):
    g = np.log1p(n)[:, None]
    return np.c_[g, P, P * g]
def switch_predict(P_rich, P_poor, n, threshold):
    return np.where(n < threshold, P_poor, P_rich)
```

ACTUAL OUTPUT

```
switching to 'bias + content' for films with fewer training ratings than the threshold (validation RMSE):
  0: 0.9155, 1: 0.9152, 3: 0.9153, 5: 0.9149, 10: 0.9149, 20: 0.9150, 50: 0.9178, 100: 0.9225
  chosen threshold 10: test 0.9054 (the blend alone 0.9061; test films below it: 617)
feature-weighted blend of item-based, ALS and content profiles: test 0.9019 (plain blend 0.9039)
  a film with   1 training ratings: weights item-based +0.62, ALS -0.03, content +0.64
  a film with  10 training ratings: weights item-based +0.52, ALS +0.27, content +0.39
  a film with 100 training ratings: weights item-based +0.39, ALS +0.65, content +0.08
  a film with 400 training ratings: weights item-based +0.31, ALS +0.89, content -0.12
   ratings  test ratings   plain  weighted
       0-5           368  1.0487    0.9930
      6-20          1105  1.0030    0.9940
     21-50          2948  0.9355    0.9329
    51-150          8413  0.8986    0.8985
      151+          7166  0.8723    0.8726
Actual output from this lesson's run.
```

What it means

Switching helped only a little: validation chose a threshold of 10 ratings, and test moved from 0.9061 to 0.9054, because only 617 of 20,000 test ratings are of such films.

The feature-weighted blend did better overall (0.9019 against 0.9039), and its weights tell the story: for a film with one rating, content gets +0.64 and ALS –0.03; at 400 ratings, ALS gets +0.89 and content –0.12. The gain is concentrated where it should be: films with 0–5 ratings went from 1.0487 to 0.9930, while films with 151+ were unchanged.

WHY CHOOSE THIS?

The mix adapts to how much is known; a few extra weights and no new model.

WHY NOT THIS? (TRADEOFFS)

Needs a meaningful feature to weight by, and more weights to fit means more validation data is needed.

4. Content inside the factor model: one model for old and new films

DEFINITION — start here, no prior knowledge assumed

Instead of blending separate models, put the content inside one: each film's factor vector is its own learned factors *plus* its content times a shared matrix Z, and a content bias is added. A film with no ratings has no own factors, but its content still gives it a factor vector, so the same model scores it. This is the idea behind LightFM, the library written by the speaker of this lesson's first talk.

It is trained by SGD exactly like Day 155's model, with the shared weights (Z and the content bias) updated from the batch average.

REAL-WORLD SCENARIO

A new employee is placed on teams by their CV at first; as their record builds up, their own track record takes over.

MATHEMATICS

$$pred = \mu + b_u + b_i + x_i \cdot a + p_u \cdot (q_i + Z^T x_i); \quad \text{new film: } b_i = 0, q_i = 0$$

IMPLEMENTATION

```
def hybrid_mf(u, i, r, X, nu, ni, k, lam, lr=0.01, lr_s=0.1, lam_s=0.001, epochs=60, batch=128,
    seed=0, track=None):
    rng = np.random.RandomState(seed); f = X.shape[1]
    seen = np.bincount(i, minlength=ni) > 0
    F = dict(mu=r.mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k) * seen[:, None],
             bu=np.zeros(nu), bi=np.zeros(ni), a=np.zeros(f), Z=0.1 * rng.randn(f, k) /
                 np.sqrt(f))
    hist = []
    for _ in range(epochs):
        o = rng.permutation(len(r))
        for s in range(0, len(o), batch):
            b = o[s:s + batch]; a, c = u[b], i[b]; xc = X[c]
            qe = F["Q"][c] + xc @ F["Z"]
            e = r[b] - (F["mu"] + F["bu"][a] + F["bi"][c] + xc @ F["a"] + (F["P"][a] * qe).sum(1))
            gp = e[:, None] * qe - lam * F["P"][a]
            gq = e[:, None] * F["P"][a] - lam * F["Q"][c]
            gz = xc.T @ (e[:, None] * F["P"][a]) / len(b) - lam_s * F["Z"]
            ga = xc.T @ e / len(b) - lam_s * F["a"]
            np.add.at(F["bu"], a, lr * (e - lam * F["bu"][a]))
            np.add.at(F["bi"], c, lr * (e - lam * F["bi"][c]))
            np.add.at(F["P"], a, lr * gp); np.add.at(F["Q"], c, lr * gq)
            F["Z"] += lr_s * gz; F["a"] += lr_s * ga
        hist.append(track(F) if track else ())
    return F, hist
def hybrid_predict(F, X, u, i):
    q = F["Q"][i] + X[i] @ F["Z"]
    return F["mu"] + F["bu"][u] + F["bi"][i] + X[i] @ F["a"] + (F["P"][u] * q).sum(1)
```

ACTUAL OUTPUT

```
Day 154's split, 100 factors, penalty 0.125: best validation 0.9212 at epoch 47 of 120 (43 s); Day 155's SGD without content
    reached 0.9229 in Day 155's run
  refit on all training ratings for 47 epochs: test 0.9104
new films (Day 156's cold split, 18947 test ratings); epochs chosen on new validation films: 8
    mean + user bias (ratings-only fallback)  1.0567
                  content profiles (Day 156)  1.0222
                factors with content inside  1.0314
          control: factors, content shuffled  1.0736
Actual output from this lesson's run.
```

What it means

On familiar films the in-model hybrid matched Day 155's learned factors: best validation 0.9212 against Day 155's 0.9229, and test 0.9104 against Day 155's 0.9105. On new films it beat the ratings-only fallback (1.0314 against 1.0567), and its shuffled-content control scored 1.0736. It did not reach Day 156's dedicated content profiles (1.0222); validation on new films stopped it after only epochs chosen on new validation films: 8 epochs, and its curve on new films is noisy.

The hand check in the exercise shows why the penalty matters here: with no penalty on the films' own factors, a toy model fitted seen films perfectly but left the content path half-learned (largest error 0.474 on unseen films); with a small penalty the content path carried the pattern (0.019). A likely reason, not tested here, for the in-model hybrid's weaker cold-start score is the same: films' own factors absorb what content could explain.

WHY CHOOSE THIS?

One model, one training loop, scores old and new films alike; the natural route to neural recommenders (Day 160).

WHY NOT THIS? (TRADEOFFS)

The balance between own factors and content depends on penalties and learning rates that are hard to tune for both old and new films at once.

5. Top-10 lists from the hybrids

DEFINITION — start here, no prior knowledge assumed

The same precision@10 as Day 154: the share of 10 recommended unrated films that the user rated 4 or 5 in the test set.

REAL-WORLD SCENARIO

A shop window is judged by whether passers-by come in, not by how well it predicts their opinion of every item inside.

ACTUAL OUTPUT

```
                              precision@10  median ratings
                ALS factors         0.0405               7
             bias + content         0.0627             100
          average of the two         0.0596              94
  factors with content inside        0.0684              99
                  popularity         0.1381             346
Actual output from this lesson's run.
```

What it means

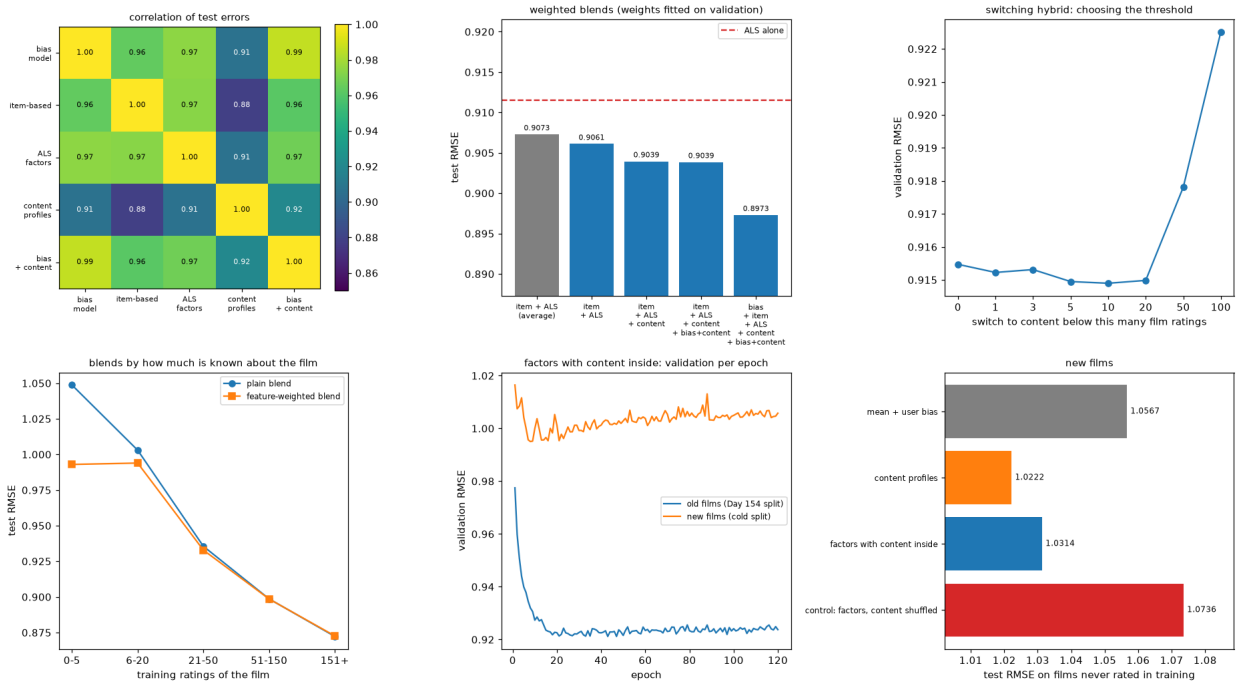
The in-model hybrid made the best personalised lists of the week so far (0.0684), ahead of bias + content (0.0627) and well ahead of ALS (0.0405), whose lists were dominated by films with a median of 7 ratings. Popularity still led by far (0.1381). Averaging ALS and bias + content (0.0596) did not beat bias + content alone: a better rating predictor is not automatically a better ranker, Day 159's subject.

WHY CHOOSE THIS?

Hybrids can fix a ranking weakness of one ingredient (ALS's taste for barely rated films).

WHY NOT THIS? (TRADEOFFS)

Blending for RMSE does not optimise the list; ranking needs its own objective and measures.



Top: correlation of the five models' test errors; test RMSE of the weighted blends; choosing the switching threshold. Bottom: plain against feature-weighted blend by film rating count; the in-model hybrid's validation per epoch on old and new films; test RMSE on new films.

Interview preparation — technical questions

Q: When does blending two recommenders help?

A: When their errors differ. Content profiles were the weakest model (0.9901) but the least correlated, so adding them took a blend from 0.9061 to 0.9039.

Q: Where do blend weights come from?

A: Least squares on predictions for ratings the models did not train on (validation here); fitting them on test would have reported 0.8963 instead of 0.8973.

Q: How do you check a blend is not overfitting?

A: Add a noise column (weight -0.0124 here), and fit on half the held-out data and score on the other half (five-model 0.9012 against three-model 0.9108).

Q: What is feature-weighted linear stacking?

A: Blend weights that are linear in features of the item; here log(1 + rating count) moved weight from content (+0.64) to ALS (+0.89) as films gained ratings.

Q: How does a model like LightFM handle new items?

A: An item's factors are its own factors plus its content times a learned matrix; a new item has only the content part.

Interview preparation — theoretical questions

Q: Why can blend weights be negative?

A: With nearly collinear inputs (bias model and bias + content, errors correlating 0.986), least squares uses their difference, with large opposite weights.

Q: Weighted, switching or feature combination?

A: Weighted is cheapest; switching handles regimes (new against old items); feature combination gives one model for both. Here the five-model weighted blend had the lowest test RMSE (0.8973); weighting by rating count improved the three-model blend (0.9019 against 0.9039).

Q: Why did the in-model hybrid trail content profiles on new films?

A: A likely reason, not tested here: films' own factors absorb what content could explain; the exercise's toy model shows the effect of the penalty.

Q: Did the Netflix Prize use hybrids?

A: Yes: the winning teams blended hundreds of models, and feature-weighted linear stacking came out of that work.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Combining recommenders works because they fail in different places. A linear blend fitted on held-out predictions beat every single model, and survived noise, leak and split checks; letting the weights depend on how many ratings a film has fixed the weakest region, films with few ratings. Putting content inside the factor model gave one model for old and new films that also made the week's best personalised top-10 lists.

- Test RMSE: ALS 0.9115, three-model blend 0.9039, feature-weighted 0.9019, five-model blend 0.8973.
- Films with 0–5 ratings: plain blend 1.0487, feature-weighted 0.9930.
- New films: fallback 1.0567, in-model hybrid 1.0314, content profiles 1.0222.
- Precision@10: in-model hybrid 0.0684, popularity 0.1381.

Suggested watch order, if starting from zero

- 1. Maciej Kula - Hybrid Recommender Systems in Python (PyData) — a PyData talk on hybrid recommenders in Python by the author of the LightFM library.
- 2. Hybrid Recommender Systems (eggie545) — a talk on hybrid recommender systems in practice.

Day 158 preview

- **Implicit feedback and cold start.** Most real systems never see star ratings, only who watched or clicked what. Day 158 turns the ratings into watched / not watched, learns from that, and looks at the other cold start: a brand-new user.